54. Capstone: Search / Graph Engine Core

54. Capstone: Search / Graph Engine Core

Phase: Phase 8 — Capstones
Estimated time: 3–5 days
Slug: `54-capstone-algorithms`

Learning objectives

- Implement indexing + query
- Prove complexity with benchmarks
- Cover edge cases with tests
- Document API and limits

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

The algorithms capstone forces you to combine data structures under realistic constraints — search, graphs, or indexes.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Clarify the problem and invariants first.
- Choose representation (adj list, heap, map indexes).
- Test edge cases heavily.
- Document complexity in comments.
- Profile only after correctness.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-capstone: && cd /tmp/gw-capstone:
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

func Index(docs []string) map[string][]int {
    inv := map[string][]int{}
    for i, d := range docs {
        inv[d] = append(inv[d], i)
    }
    return inv
}

func main() { fmt.Println(Index([]string{"a", "b", "a"})) }
```

Run it:

```
go run .
```

Expected output:

```
map[a:[0 2] b:[1]]
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Build an inverted index.
2. Query for documents containing a term.
3. Add a simple rank by term frequency.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Mission

Build an in-memory search or routing core:

Option A: inverted index for document search (tokenizeize → postings → rank) Option B: weighted graph shortest path service (Dijkstra)

Milestones

1. Data model + ingest
2. Query algorithm + ranking/path
3. Table tests for correctness
4. Benchmarks at 10k+ entities
5. Optional: concurrent read-safe index with RWMutex

Rubric (excellent)

- Stated complexities match implementation

- Deterministic results
- Clean exports and docs
- Benchmarks show scaling behavior

Failure modes to watch while coding

- Unbenchmarked “optimizations”
- Global mutable state without sync
- No empty-query handling

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Same CS; Go types force explicit structures.
- dict.setdefault ≈ append to map slices.
- Prove correctness before optimizing.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Write a one-page design note before coding the capstone.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Capstone: Search / Graph Engine Core” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Unbenchmarked “optimizations”
- Global mutable state without sync
- No empty-query handling

Exercises

Work in `exercises/54-capstone-algorithms/`. Read that folder's `README.md` before you code.

What you will build

Implement the search/graph engine pieces the tests name.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/54-capstone-algorithms/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Which option did you pick and why?
2. What is your query complexity?
3. How test concurrency?

Further reading

- https://pkg.go.dev/container/heap